

Modul: M217 IT-Sicherheit

Prüfungsprojekt: Sommersemester 2026

Gruppe: Gruppe A

Mitglieder:

- Dennis Pfeil
- Tim Ziemann
- Ahmad Mahmoud

Repository: <https://github.com/Ahmad-Mtr/it-sicherheit>

1. Systemüberblick

Das System verwaltet und analysiert Software-Schwachstellen. Konkrete Schwachstellen werden als **CVE** (Common Vulnerabilities and Exposures) gespeichert, allgemeine Schwachstellentypen über die **CWE** (Common Weakness Enumeration) abgebildet.

Technische Basis sind eine PostgreSQL-Datenbank und eine REST-API auf Basis von Bun und Elysia. Der Datenbankzugriff erfolgt über Drizzle ORM.

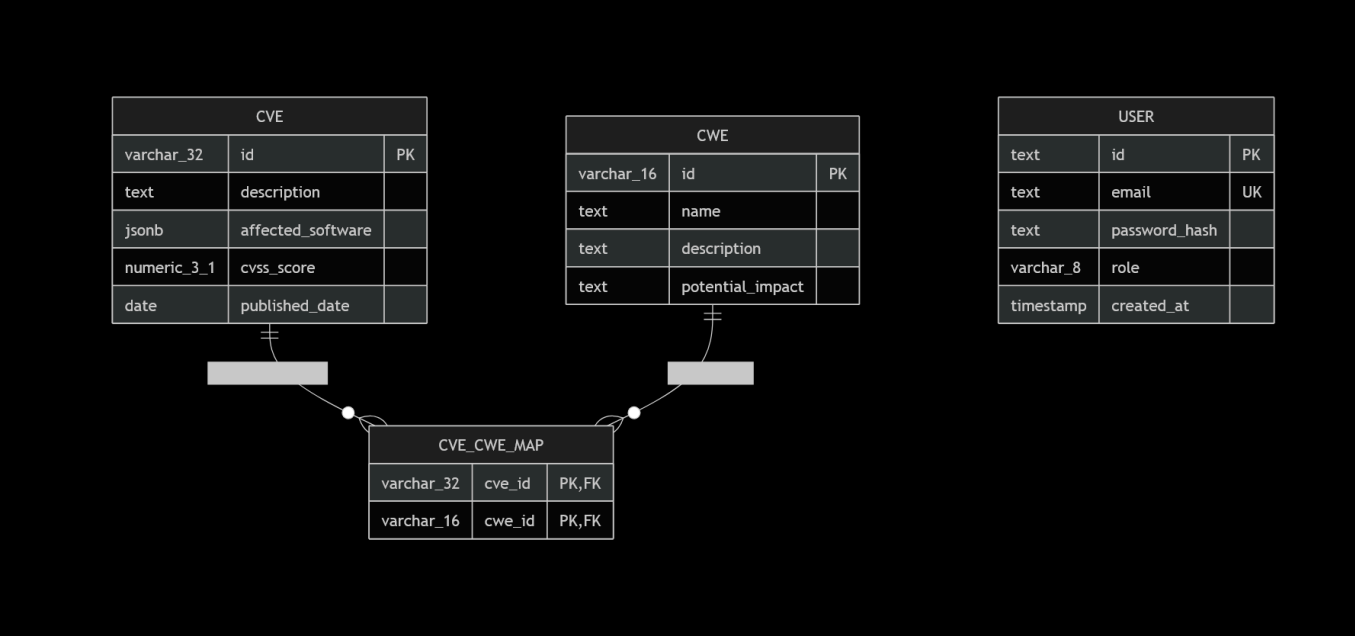
Teilaufgabe	Inhalt	Umsetzung
A	Datenmodellierung und API	<code>packages/db/src/schema.ts</code> , <code>apps/api/src/routes/</code>
B	Vulnerability Analysis	<code>apps/api/src/routes/scan.ts</code> , <code>packages/shared/src/index.ts</code>
C	API Hardening nach OWASP	<code>apps/api/src/lib/auth.ts</code> , <code>rateLimit.ts</code> , <code>errorHandler.ts</code>

Das Projekt ist als Monorepo organisiert:

Verzeichnis	Inhalt
<code>apps/api</code>	Elysia-Server mit Routen, Auth, Rate-Limiting und Fehlerbehandlung
<code>packages/db</code>	Drizzle-Schema, Migrationen und Seed-Skript
<code>packages/shared</code>	Gemeinsame Typen und Versionsvergleichslogik
<code>inventory.json</code>	Beispiel-Inventar für den Scanner

Die Versionsvergleichslogik liegt bewusst in einem eigenen Paket, da sie sowohl vom Such-Endpunkt als auch vom Scanner verwendet wird. Bei einer doppelten Implementierung könnten beide unterschiedliche Ergebnisse liefern.

2. ER-Diagramm der Datenbank



2.1 Tabellen

Tabelle	Beschreibung
cve	Speichert konkrete Software-Schwachstellen
cwe	Speichert allgemeine Schwachstellentypen
cve_cwe_map	Verknüpft CVEs und CWEs
user	Speichert Benutzer für Authentifizierung und Rollenprüfung

Tabelle cve

Feld	Datentyp	Bedeutung
id	varchar(32)	CVE-ID und Primärschlüssel
description	text	Beschreibung der Schwachstelle
affected_software	jsonb	Betroffene Produkte und Versionsbereiche
cvss_score	numeric(3,1)	CVSS Base Score

Feld	Datentyp	Bedeutung
published_date	date	Veröffentlichungsdatum

Tabelle cwe

Feld	Datentyp	Bedeutung
id	varchar(16)	CWE-ID und Primärschlüssel
name	text	Name des Schwachstellentyps
description	text	Ausführliche Beschreibung
potential_impact	text	Mögliche Auswirkungen

Tabelle cve_cwe_map

Feld	Bedeutung
cve_id	Fremdschlüssel auf cve.id
cwe_id	Fremdschlüssel auf cwe.id

Tabelle user

Feld	Bedeutung
id	Benutzer-ID
email	Eindeutige E-Mail-Adresse
password_hash	Hash des Benutzerpassworts
role	Rolle des Benutzers
created_at	Erstellungszeitpunkt

Die Benutzertabelle besitzt keine Beziehung zu CVE oder CWE. Sie dient ausschließlich der Authentifizierung aus Teilaufgabe C.

2.2 Modellierungsentscheidungen

n:m-Beziehung zwischen CVE und CWE. Eine Spalte `cwe_id` in der Tabelle `cve` wäre fachlich nicht korrekt, da CVEs regelmäßig mehreren Schwachstellentypen zugeordnet sind. CVE-2014-0160 ist beispielsweise sowohl CWE-125 als auch CWE-119 zugeordnet. Der

zusammengesetzte Primärschlüssel aus `cve_id` und `cwe_id` verhindert doppelte Zuordnungen. Beide Fremdschlüssel verwenden `ON DELETE CASCADE`.

Katalog-IDs als Primärschlüssel. Die Bezeichner `CVE-2014-0160` und `CWE-89` sind global eindeutig und stabil. Ein zusätzlicher fortlaufender Schlüssel brächte keinen Nutzen. Zugleich ist dies die Voraussetzung für den idempotenten Import (Abschnitt 3.2).

affected_software als JSONB. Eine CVE kann mehrere Produkte mit jeweils mehreren Versionsbereichen betreffen:

```
[
  { "product": "lodash", "versions": ["<4.17.21"] },
  { "product": "express", "versions": [">=4.0.0 <4.18.2"] }
]
```

Eine eigene Tabelle für Versionen würde die Bereiche nicht auflösen. Ein Wert wie `>=1.0.1 <1.0.1g` bliebe auch dort eine Zeichenkette, da die Menge der enthaltenen Versionen unbegrenzt ist. Die JSONB-Variante erlaubt stattdessen eine Vorfilterung über `jsonb_array_elements` direkt in der Datenbank.

cvss_score als numeric(3,1). CVSS Base Scores sind auf eine Nachkommastelle definiert. Zu beachten ist, dass der PostgreSQL-Treiber `numeric`-Werte als Zeichenkette zurückgibt. Die Sortierung konvertiert deshalb explizit nach `Number`, da sonst lexikografisch sortiert würde und `9.8` über `10.0` läge.

3. Begründung der API-Design-Entscheidungen

3.1 Struktur und Zugriffsmodell

Die API ist nach fachlichen Ressourcen gegliedert.

Route	Verantwortlichkeit	Zugriff
/api/cves	Abfrage und Verwaltung von CVEs	Lesen frei
/api/cwes	Abfrage und Verwaltung von CWEs	Lesen frei
/api/import	Batch-Import von CVE- und CWE-Daten	Authentifizierung
/api/auth	Anmeldung und Erzeugung von JWTs	frei
/api/scan	Analyse einer Software-Inventarliste	frei

Route	Verantwortlichkeit	Zugriff
/health	Erreichbarkeitsprüfung	frei
/docs	Swagger-Dokumentation	frei

Löschooperationen erfordern zusätzlich die Rolle `admin`.

Lesende Zugriffe sind bewusst offen: CVE- und CWE-Kataloge sind öffentliche Daten, ihre Vertraulichkeit zu schützen wäre nicht sinnvoll. Schützenswert ist die Integrität des Datenbestands, denn wer Schwachstellendaten verändern kann, kann ein verwundbares System als sicher erscheinen lassen. Die Zugriffskontrolle liegt deshalb auf den schreibenden Operationen.

3.2 Import-Schnittstelle

Die Import-Endpunkte verarbeiten bis zu 500 Datensätze pro Request und schreiben diese in einer einzigen `INSERT`-Anweisung. Das reduziert HTTP-Overhead und Datenbankoperationen und eignet sich für automatisierte Importe größerer Kataloge.

Bereits vorhandene IDs werden per Upsert aktualisiert. Importe können dadurch wiederholt werden, ohne doppelte Primärschlüssel zu erzeugen. Dies entspricht dem realen Betrieb, da Schwachstellenkataloge regelmäßig aktualisiert werden und CVSS-Scores nachträglich korrigiert werden.

3.3 Abfrage-Schnittstelle

Listenabfragen unterstützen `limit` und `offset` und liefern die Metadaten `total`, `limit`, `offset` und `hasMore`. Der Wert `limit` ist auf 100 begrenzt, da eine Anfrage wie `?limit=999999` sonst den gesamten Datenbestand in einer Antwort anfordern könnte.

Ergebnisse werden immer absteigend nach CVSS-Score sortiert. Eine kritische Schwachstelle soll nicht erst auf einer späteren Ergebnisseite erscheinen.

Zu jeder CVE werden die verknüpften CWEs direkt mitgeliefert. Sie werden nicht einzeln nachgeladen, sondern für die gesamte Ergebnisseite in einer gemeinsamen Abfrage mit `WHERE cve_id IN (...)` geladen.

3.4 Hybride Produkt- und Versionsprüfung

Die Produktnamen werden in PostgreSQL gefiltert, die Versionsbereiche anschließend in der Anwendung geprüft:

```
matchesVersionRange(version, range)
```

Der Grund liegt in der Semantik des Vergleichs. Die Prüfung, ob `1.0.1f` im Bereich `>=1.0.1 <1.0.1g` liegt, folgt den Regeln der semantischen Versionierung, die SQL nicht abbilden kann. Ein reiner Zeichenkettenvergleich würde `1.0.10` unterhalb von `1.0.9` einordnen.

Die Datenbank reduziert damit früh die Kandidatenmenge, während die komplexere Prüfung in einer zentralen und wiederverwendbaren Funktion bleibt.

3.5 Eingabevalidierung

Die API validiert Request-Bodys, Query- und Pfadparameter über Elysia-Schemas. Geprüft werden Pflichtfelder, Feldlängen, Zahlenbereiche und maximale Array-Größen. Für Importe gelten zusätzlich feste Formate:

Feld	Regel
CVE-ID	<code>^CVE-\d{4}-\d{4,}\$</code>
CWE-ID	<code>^CWE-\d+\$</code>
cvssScore	Wertebereich 0 bis 10

Ungültige Eingaben werden verworfen, bevor sie die Geschäftslogik oder die Datenbank erreichen. Dieselben Schemas erzeugen die Swagger-Dokumentation unter `/docs`, sodass Dokumentation und Validierung nicht auseinanderlaufen.

4. Security-Matrix nach OWASP

OWASP-Kategorie	Technische Umsetzung	Schutzwirkung	Datei
A01: Broken Access Control	JWT-Authentifizierung über Bearer-Tokens mit zwei Stunden Gültigkeit; <code>requireAuth()</code> schützt die Import-Endpunkte; <code>requireRole("admin")</code> schützt Löschoperationen; die Rolle stammt aus dem	Verhindert anonyme Datenimporte und beschränkt administrative Löschooperationen auf Administratoren	<code>lib/auth.ts</code> , <code>routes/import.ts</code> , <code>routes/cves.ts</code> , <code>cwes.ts</code>

OWASP-Kategorie	Technische Umsetzung	Schutzwirkung	Datei
	signierten Token, nicht aus dem Request-Body		
A01: Broken Access Control	Passwörter werden ausschließlich als Hash gespeichert und mit <code>Bun.password.verify()</code> geprüft; fehlgeschlagene Logins antworten einheitlich mit <code>401</code>	Verhindert Klartext-Passwörter in der Datenbank und die Enumeration existierender Konten	<code>routes/auth.ts</code>
A03: Injection	Datenbankabfragen über Drizzle ORM mit <code>eq()</code> , <code>inArray()</code> und parametrisierter SQL-Interpolation; auch die dynamischen JSONB-Filter des Scanners nutzen Bind-Parameter; keine Stringverkettung mit Benutzereingaben	Verhindert, dass Benutzereingaben als ausführbarer SQL-Code interpretiert werden	<code>routes/scan.ts</code> , <code>routes/cves.ts</code>
A04: Insecure Design / Rate Limiting	Globales Limit von 100 Requests pro 60 Sekunden je Client-IP; striktes Limit von 10 Requests pro 60 Sekunden für Scanner und Import; zusätzlich 500 Objekte pro Import, 1000 Pakete pro Scan, 100 Einträge pro Seite	Reduziert automatisierten Missbrauch, Lastspitzen und einfache Denial-of-Service-Angriffe	<code>lib/rateLimit.ts</code> , <code>pagination.ts</code>
A05: Security Misconfiguration	Globaler Error-Handler mit generischen Meldungen und Request-ID; keine Stack-Traces oder SQL-Details in Client-Antworten; die API startet nicht ohne gesetztes <code>JWT_SECRET</code> , ein Standardwert existiert bewusst nicht	Verhindert die Offenlegung interner Systemdetails und den Betrieb mit unsicherer Konfiguration	<code>lib/errorHandler.ts</code> <code>lib/auth.ts</code>

Ergänzende Hinweise:

- Fehlender oder ungültiger Token führt zu `401`, ein gültiger Token mit falscher Rolle zu `403`.
- Der einzige Aufruf von `sql.raw()` verarbeitet ausschließlich fest im Quelltext hinterlegte Spaltennamen für die `EXCLUDED`-Pseudotabelle des Upserts, keine Benutzereingaben.
- Rate-Limiting und Mengenbegrenzung ergänzen sich: Ersteres begrenzt die Anzahl der Requests, Letzteres den Aufwand eines einzelnen Requests.
- Die Datei `.env` ist über `.gitignore` von der Versionsverwaltung ausgenommen, versioniert wird nur `.env.example`.

Beispiel einer Fehlerantwort:

```
{  
  "error": "Internal server error",  
  "requestId": "019f991b-5e77-7000-a266-6d13ac52399f"  
}
```

Die vollständigen Fehlerdetails werden ausschließlich serverseitig protokolliert. Die Request-ID erlaubt die Zuordnung zum Log-Eintrag, ohne interne Informationen offenzulegen.

5. Vulnerability Scanner

Der Scanner nimmt eine Liste installierter Software entgegen und prüft sie gegen den CVE-Bestand.

Verarbeitung

1. Validierung der Eingabe
2. Normalisierung der Produktnamen und Entfernung identischer Pakete
3. Datenbankabfrage nach passenden Produktnamen
4. Vergleich der Versionen mit den betroffenen Versionsbereichen
5. Ermittlung der verknüpften CWE-Datensätze
6. Sortierung nach CVSS-Score, bei Gleichstand nach CVE-ID
7. Erzeugung der Ergebnis- und Metadatenstruktur

Die Normalisierung entfernt Leerzeichen und wandelt Produktnamen in Kleinbuchstaben, sodass `Lodash` und `lodash` identisch behandelt werden. Das eingereichte Inventar wird

nicht gespeichert, da eine Liste installierter Software ein wertvolles Ziel für die Ausspähung eines Systems ist.

Anfrage

```
POST /api/scan
```

```
{
  "packages": [
    { "name": "jsonwebtoken", "version": "8.5.1" },
    { "name": "lodash", "version": "4.17.20" }
  ]
}
```

Antwort

```
{
  "data": [
    {
      "id": "CVE-2023-10004",
      "description": "Example vulnerability",
      "cvssScore": 9.8,
      "publishedDate": "2023-01-01",
      "affectedSoftware": [
        {
          "product": "jsonwebtoken",
          "versions": [
            "<9.0.0"
          ]
        }
      ],
      "matchedPackages": [
        {
          "name": "jsonwebtoken",
          "version": "8.5.1",
          "affectedRanges": [
            "<9.0.0"
          ]
        }
      ],
      "cwes": [
        {
```

```

        "id": "CWE-287",
        "name": "Improper Authentication",
        "description": "Authentication is not performed correctly.",
        "potentialImpact": "Unauthorized access"
    }
]
}
],
"meta": {
    "submittedPackages": 2,
    "scannedPackages": 2,
    "vulnerablePackages": 1,
    "vulnerabilitiesFound": 1
}
}

```

Die Ergebnisse sind absteigend nach CVSS-Score sortiert, die kritischste Schwachstelle steht zuerst. Die Priorisierung erfolgt serverseitig, damit ein Client die Reihenfolge nicht verlieren kann.

Das Feld `matchedPackages` macht nachvollziehbar, welches Paket die CVE ausgelöst hat. Die mitgelieferten CWE-Daten zeigen den Typ des Fehlers: Die Angabe `CVE-2014-0160`, `CVSS 7.5` allein lässt nicht erkennen, ob ein Speicherfehler oder eine Fehlkonfiguration vorliegt.

Die Metadaten sind notwendig, weil eine leere Ergebnisliste sonst mehrdeutig wäre. Sie könnte bedeuten, dass nichts gefunden wurde oder dass nichts geprüft wurde.

6. Testdaten

Der Datenbestand wird über `packages/db/src/seed.ts` erzeugt.

Anforderung	Gefordert	Vorhanden
CWE-Katalog	20	32
CVE-Katalog	50	59
Testinventar	10 Pakete	10

Der CVE-Bestand besteht aus 14 manuell gepflegten Einträgen, darunter reale Schwachstellen wie CVE-2014-0160 (Heartbleed), CVE-2022-3602 (SpookySSL) und CVE-2016-0800 (DROWN), sowie 45 generierten Einträgen über 20 Produkte. Die Generierung

verwendet einen deterministischen Zufallszahlengenerator mit festem Startwert, sodass jeder Durchlauf denselben Datenbestand erzeugt.

Zusätzlich werden zwei Benutzerkonten angelegt:

E-Mail	Rolle	Berechtigung
admin@example.com	admin	Import und Löschen
user@example.com	user	Import

Geprüfte Beispiele:

Produkt	Version	Betroffener Bereich	Erwartetes Ergebnis
jsonwebtoken	8.5.1	<9.0.0	verwundbar
lodash	4.17.20	<4.17.21	verwundbar
axios	1.5.0	<1.6.0	verwundbar
express	4.17.1	<4.18.2	verwundbar
openssl	1.0.1f	>=1.0.1 <1.0.1g	verwundbar
lodash	99.0.0	<4.17.21	kein Treffer

Der Fall `openssl 1.0.1f` prüft einen zweiseitig begrenzten Versionsbereich, der Fall `lodash 99.0.0` die korrekte Nichtmeldung einer nicht betroffenen Version. Die zugehörigen Testbefehle sind in `SETUP.md` dokumentiert.

7. Abgrenzung

Die geforderten Sicherheitsmaßnahmen sind umgesetzt. Für einen produktiven Betrieb wären weitere Maßnahmen erforderlich, die nicht zu den Mindestanforderungen des Prüfungsprojekts gehören:

- Beschränkung der erlaubten CORS-Ursprünge, derzeit ist CORS allgemein aktiviert
- Speicherung der Rate-Limit-Zähler in einem zentralen System wie Redis, da sie aktuell pro Instanz im Arbeitsspeicher liegen
- Refresh-Tokens, Token-Rotation und Widerrufsmöglichkeiten
- sichere Verwaltung des produktiven `JWT_SECRET`
- strukturierte Protokollierung, Monitoring und Audit-Logging für Import- und Löschoperationen

- TLS-Terminierung
- automatisierte Integrations- und Sicherheitstests
- GIN-Index auf `affected_software` bei deutlich größeren Datenmengen

Auf Refresh-Tokens wurde bewusst verzichtet, da sie durch Rotation und Widerruf eine eigene Angriffsfläche mitbringen. Für den Projektumfang ist die feste Gültigkeitsdauer von zwei Stunden ausreichend.

8. Fazit

Das System bildet CVEs und CWEs in einem relationalen Datenmodell ab und stellt sie über eine REST-API mit Batch-Import und flexiblen Abfragen bereit. Der Vulnerability Scanner vergleicht installierte Softwarepakete gegen diesen Bestand, priorisiert die Ergebnisse nach CVSS-Score und gibt zu jeder CVE die verknüpfte CWE aus.

Die geforderten OWASP-Maßnahmen wurden über JWT-Authentifizierung, rollenbasierte Zugriffskontrolle, parametrisierte Datenbankabfragen, Eingabevalidierung, zweistufiges Rate-Limiting und eine zentrale generische Fehlerbehandlung umgesetzt. Damit sind die Anforderungen der drei Projektmodule erfüllt.

9. Quellen

1. Aufgabenstellung: *IT-Sicherheit – Prüfungsprojekt SS 2026*
2. Projekt-Repository: <https://github.com/Ahmad-Mtr/it-sicherheit>
3. Datenbankschema: <https://github.com/Ahmad-Mtr/it-sicherheit/blob/main/packages/db/src/schema.ts>
4. API-Einstiegspunkt: <https://github.com/Ahmad-Mtr/it-sicherheit/blob/main/apps/api/src/index.ts>
5. Authentifizierungslogik: <https://github.com/Ahmad-Mtr/it-sicherheit/blob/main/apps/api/src/lib/auth.ts>
6. Fehlerbehandlung: <https://github.com/Ahmad-Mtr/it-sicherheit/blob/main/apps/api/src/lib/errorHandler.ts>
7. Rate-Limiting: <https://github.com/Ahmad-Mtr/it-sicherheit/blob/main/apps/api/src/lib/rateLimit.ts>
8. CVE-Routen: <https://github.com/Ahmad-Mtr/it-sicherheit/blob/main/apps/api/src/routes/cves.ts>

9. CWE-Routen: <https://github.com/Ahmad-Mtr/it-sicherheit/blob/main/apps/api/src/routes/cwes.ts>
10. Import-Routen: <https://github.com/Ahmad-Mtr/it-sicherheit/blob/main/apps/api/src/routes/import.ts>
11. Scanner-Route: <https://github.com/Ahmad-Mtr/it-sicherheit/blob/main/apps/api/src/routes/scan.ts>
12. Versionsvergleichslogik: <https://github.com/Ahmad-Mtr/it-sicherheit/blob/main/packages/shared/src/index.ts>
13. Seed-Skript mit Testdaten: <https://github.com/Ahmad-Mtr/it-sicherheit/blob/main/packages/db/src/seed.ts>